TRAFFIC LIGHT DETECTION USING ON KRIA KV260

Name: C. INDHU REDDY

Roll Number: S20261020003

ABSTRACT

This project presents a traffic light detection system using the YOLOv3-Tiny model on the Kria KV260 FPGA board. The system detects different traffic light signals in an image and identifies them based on their types, such as Red, Green, Yellow, RedLeft, RedRight, GreenLeft, GreenRight and off. The BOSCH Small Traffic Light Dataset is used for traffic light detection. Vitis AI tools are used to prepare the model for the KV260 platform, and the DPU helps in processing the model using FPGA acceleration. This project demonstrates the use of FPGA technology for traffic light detection and its application in smart vehicles and road safety systems.

1. INTRODUCTION

Traffic lights play an important role in controlling vehicles on roads. Identifying traffic signals correctly helps to improve road safety and avoid accidents. In this project, YOLOv3-Tiny is used to detect traffic lights from images. The project is implemented using the Kria KV260 FPGA board. The board supports AI applications and helps to process object detection tasks using hardware acceleration. The BOSCH Traffic Light Dataset is used in this project. It contains different types of traffic light signals, such as Red, Green, Yellow and other classes. Vitis AI tools are used to prepare the model for the KV260 board.

1.1 Objective

- To detect traffic lights from input images.
- To identify the traffic light colour and type.
- To display bounding boxes with the corresponding class names.
- To process traffic light images using the KV260 FPGA board.

1.2 Applications

- Autonomous Vehicles
- Driver Assistance Systems
- Smart Traffic Management
- Road Safety
- Intelligent Transportation

2. SYSTEM REQUIREMENTS

2.1 Hardware Requirements

- Kria KV260 FPGA Board
- MicroSD Card
- Power Supply
- Monitor
- USB Cable

2.2 Software Requirements

- Ubuntu 22.04
- Docker
- Vitis AI 2.5
- TensorFlow 2.8
- YOLOv3-Tiny
- Python
- OpenCV
- Darknet

3. KRIA KV260 FPGA BOARD

The Kria KV260 is an FPGA development board designed for AI and computer vision applications. It is developed by AMD Xilinx. It can perform image processing and object detection using hardware acceleration.

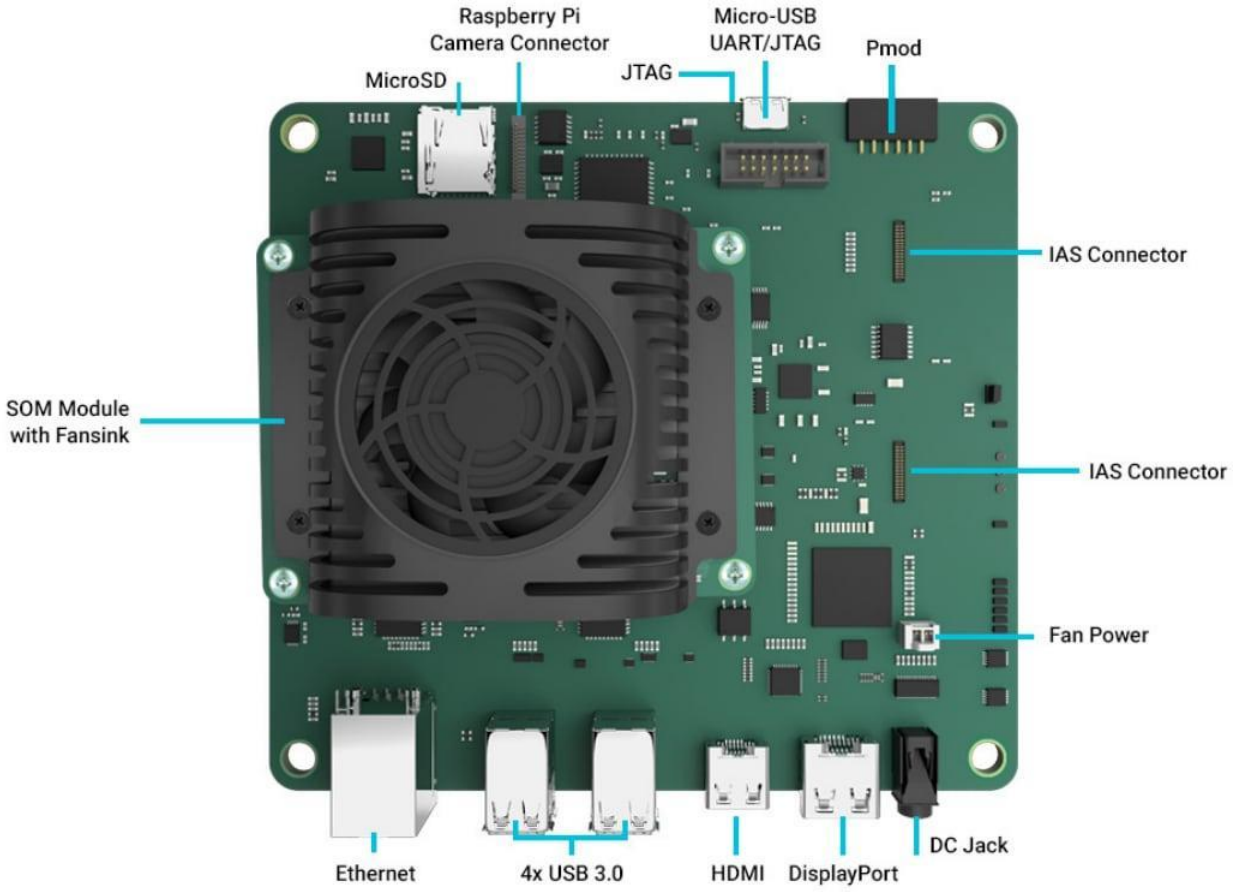


Fig 3.1: Kria KV 260 Board

1. Processing System (PS): Controls the board and runs the Linux operating system.
2. Programmable Logic (PL): Performs hardware-based processing operations.

3.1 Working of KV260

The input image is given to the system. The YOLOv3-Tiny model processes the image, and the DPU helps to perform the required operations. Finally, the detected traffic lights are displayed with bounding boxes and their class names.

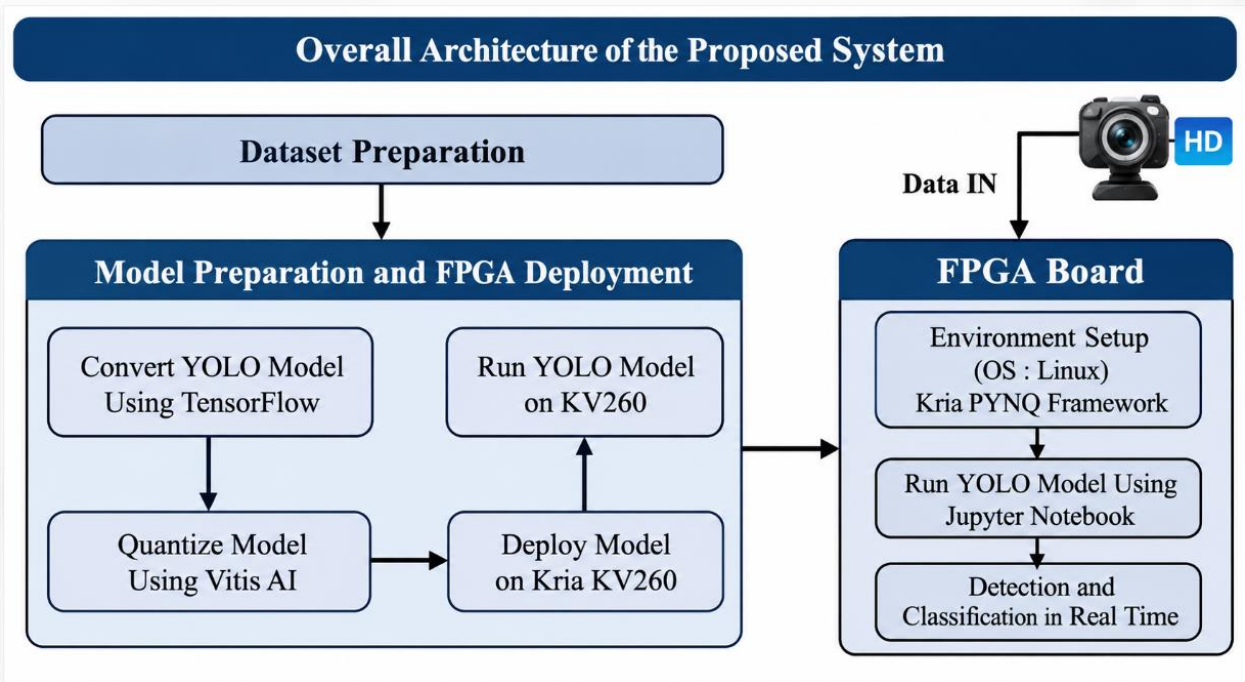


Fig 3.2 : Proposed model block diagram

4. SYSTEM SETUP

4.1 Host System Preparation

- Update the Ubuntu system using the following command :

```
sudo apt update && sudo apt upgrade -y
```

4.2 Vitis AI Environment Setup

Docker is used to run the Vitis AI development environment. The Vitis AI 2.5 Docker image is used for model preparation and compilation.

The following steps are performed:

1. Install and configure Docker in Ubuntu.
2. Download the Vitis AI 2.5 Docker image.
3. Start the Vitis AI Docker container.
4. Activate the TensorFlow 2 environment.
5. Verify the TensorFlow installation.

4.3 Model Deployment on Kria KV260

The compiled YOLOv3-Tiny model and the required files are transferred from the host system to the Kria KV260 board.

Connect to the KV260 board:

```
ssh root@<KV260_IP>
```

Check the transferred files:

```
ls -lh /home/root/
```

The following files are used:

- yolov3_tiny_bosch.xmodel – Compiled YOLOv3-Tiny model.
- yolov3-tiny-bosch.cfg – Model configuration file.
- voc-bosch.names – Traffic-light class names.
- t1.png – Input test image.

4.4 Traffic Light Detection

The DPU accelerator on the KV260 board is checked before performing traffic light detection. The following commands are used to check the board, model information and model performance.

Check the DPU and board information:

```
xdputil query
```

Check the compiled model information:

```
xdputil query /home/root/yolov3_tiny_bosch.xmodel
```

Run the model benchmark:

```
xdputil benchmark /home/root/yolov3_tiny_bosch.xmodel 1
```

The input image is provided to the traffic light detection program. The YOLOv3-Tiny model processes the image using the DPU accelerator.

Run traffic light detection:

```
python3 bosch_detect.py --model /home/root/yolov3_tiny_bosch.xmodel --input /home/root/t1.png --output /home/root/output.png
```

In this command:

- --model specifies the compiled YOLOv3-Tiny model.
- --input specifies the input image path.
- --output specifies the path for saving the detection result.

The detection results are decoded to identify traffic-light locations and their corresponding classes. Bounding boxes, class names and confidence scores are displayed on the output image.

4.5 Output

The detection results are saved as an output image containing bounding boxes, corresponding class labels and confidence scores.

1. Display the generated output image on the KV260 board:

```
display /home/root/output.png
```

2. Transfer the output image from the KV260 board to the host PC:

Run this command in the Ubuntu terminal on your PC:

```
scp root@<KV260_IP>:/home/root/output.png /mnt/c/Users/chepa/Downloads/
```

The output image is copied from the KV260 board to the Downloads folder on the host PC. The generated image shows detected traffic lights with their corresponding bounding boxes and class labels.

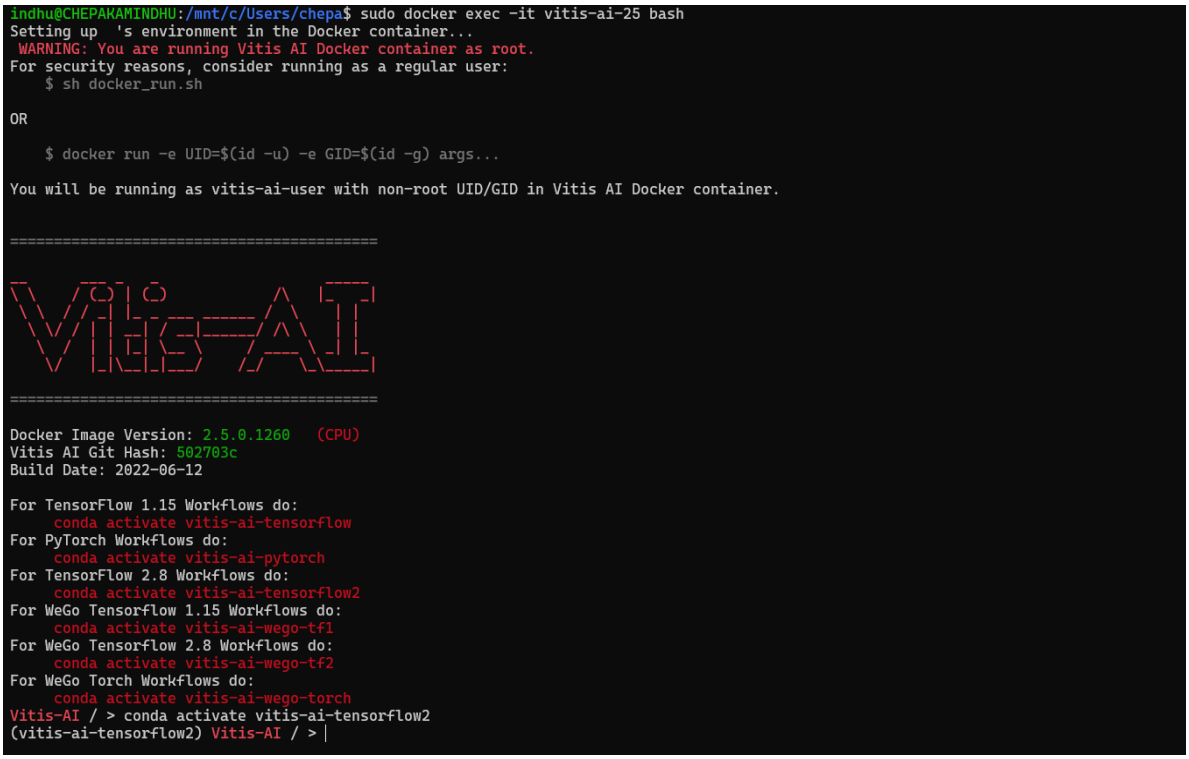


Fig. 4.1: Activation of TensorFlow 2 Environment in Vitis AI Docker Container

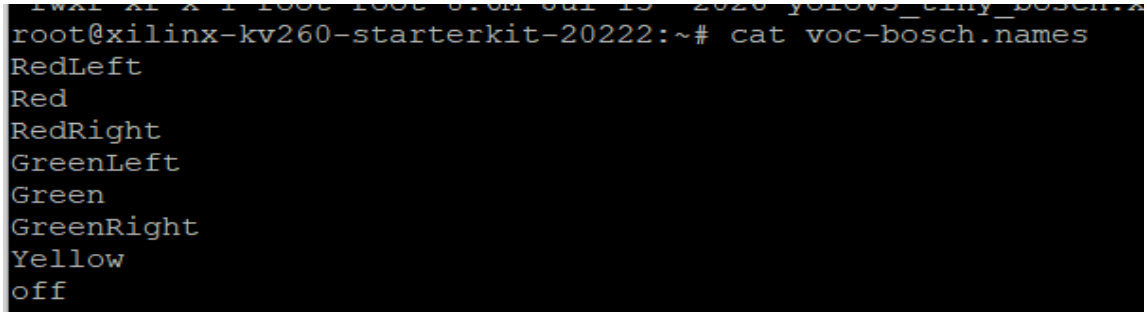


Fig. 4.2: Traffic Light Class Names Used in YOLOv3-Tiny Model

5. RESULTS AND DISCUSSION

5.1 Input Image

An input image containing traffic lights is provided to the detection system for processing. The image is used to identify different traffic light signals using the YOLOv3-Tiny model. The input image is resized to 416 × 416 pixels before being processed by the model. This image serves as the input for traffic light detection and classification on the Kria KV260 FPGA board.



Fig 5.1: Input Image



Fig 5.2 : Output Image

5.2 Traffic Light Detection Output

After processing the input image, the YOLOv3-Tiny model identifies traffic lights present in the image. The detected objects are marked with bounding boxes, and their corresponding class names are displayed. The system can identify different traffic light categories, including Red, Green, Yellow, RedLeft, RedRight, GreenLeft, GreenRight and off.



Fig 5.3: Output image2



Fig 5.4: Output image3



Fig 5.5: Output image4

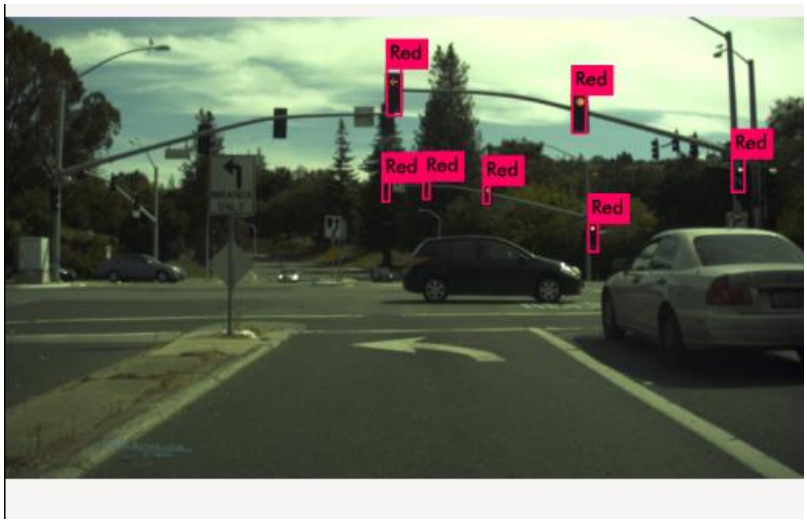


Fig 5.6: Output image5

5.3 Advantages of the Proposed System

The proposed traffic light detection system offers several advantages:

- Detects different types of traffic light signals.
- Identifies traffic light colours and directions.
- Displays detected objects with bounding boxes and class labels.
- Uses FPGA acceleration for model processing.
- Provides a compact solution for edge-based object detection.
- Can be applied in smart vehicles and intelligent transportation systems.
- Supports road safety and driver assistance applications.

6. CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This project presents a traffic light detection system using the YOLOv3-Tiny model on the Kria KV260 FPGA board. The system is designed to detect and classify different traffic light signals using the BOSCH Small Traffic Light Dataset. Vitis AI tools are used for model preparation and deployment, while the DPU accelerator supports hardware-based processing. The project demonstrates the use

of FPGA technology in computer vision applications and its potential to support intelligent transportation and road safety systems.

6.2 Future Scope

The proposed system can be further improved in several ways:

- Extend traffic light detection to real-time video streams.
- Improve detection performance in different lighting and weather conditions.
- Integrate the system with vehicle driver assistance applications.
- Explore additional optimization techniques to improve processing speed.
- Combine traffic light detection with other road object detection systems.

7. REFERENCES

Rashed Al Amin, Mehrab Hasan, Veit Wiese, and Roman Obermaisser, "FPGA-Based Real-Time Object Detection and Classification System Using YOLO for Edge Computing," IEEE Access, vol. 12, pp. 73268–73278, 2024.

https://doi.org/10.1109/ACCESS.2024.3404623